

Contents

1	DD09: 認可ヘルパ	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	1
1.4.1	3.1 認可ヘルパ実装	1
1.4.2	3.2 Principal 構築（参照のみ）	2
1.4.3	3.3 オーナー境界（NFR-301）	2
1.4.4	3.4 不正アクセスの監査記録	2
1.5	4. 関連設計	3
1.6	5. テスト観点	3
1.6.1	5.1 オーナー境界によるデータ分離検証	3
1.6.2	5.2 その他観点	3
1.7	6. 未確定事項・確認事項	3

1 DD09: 認可ヘルパ

1.1 0. 文書情報

項目	内容
文書名	DD09: 認可ヘルパ
詳細設計 ID	DD09
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	NFR-301（オーナー境界によるデータ分離） / FR-007 / AC-017
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	NFR-301	オーナー境界によるデータ分離
機能	FR-007	認証セッション + 認可
ライブラリ	app/workers/main-api/src/lib/authz.ts	requireTenant / requireProjectRole / requireInquiry / requireOwner

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 10.6	認可ヘルパ	requireTenant / requireProjectRole / requireInquiry / requireOwner

1.4 3. 詳細設計本文

1.4.1 3.1 認可ヘルパ実装

```
// app/workers/main-api/src/lib/authz.ts
export type ProjectRole = 'admin' | 'member';

export function requireTenant(c: Context, expectedTenantId: string) {
  const principal = c.get('principal');
  if (principal.ownerAccountId !== expectedTenantId) throw new HTTPException(404); // 404 偽装
}

export function requireOwner(c: Context) {
```

```

    const principal = c.get('principal');
    if (!principal.isOwner) throw new HTTPException(403); // E-AUTHZ-OWNER-ONLY
}

export async function requireProjectRole(c: Context, projectId: string, minRole: ProjectRole) {
    const principal = c.get('principal');
    const project = await c.env.DB.prepare(
        `SELECT owner_account_id FROM projects WHERE id = ?1`
    ).bind(projectId).first<{ owner_account_id: string }>();
    if (!project) throw new HTTPException(404);
    if (project.owner_account_id !== principal.ownerAccountId) throw new HTTPException(404); // 4

    if (principal.isOwner) return; // オーナーは全プロジェクト bypass(暗黙の admin)

    const granted = principal.projectRoles.get(projectId);
    if (!granted) throw new HTTPException(404); // E-AUTHZ-PROJECT-DENIED(404 偽装)
    if (minRole === 'admin' && granted !== 'admin') throw new HTTPException(403); // E-AUTHZ-FORB
    // minRole='member' は admin/member いずれでも通過
}

export async function requireInquiry(c: Context, inquiryId: string) {
    const principal = c.get('principal');
    const row = await c.env.DB.prepare(
        `SELECT owner_account_id, project_id FROM inquiries WHERE id = ?1`
    ).bind(inquiryId).first<{ owner_account_id: string; project_id: string }>();
    if (!row) throw new HTTPException(404);
    if (row.owner_account_id !== principal.ownerAccountId) throw new HTTPException(404);
    await requireProjectRole(c, row.project_id, 'member');
}

export function requireRole(...roles: Role[]): MiddlewareHandler {
    return async (c, next) => {
        const principal = c.get('principal');
        if (!roles.includes(principal.role)) throw new HTTPException(403);
        await next();
    };
}

```

1.4.2 3.2 Principal 構築 (参照のみ)

`c.get('principal')` で取得する Principal の構築フローは [../02_基本設計/08_認証・認可設計.md](#) を正本とする。Principal には以下を含む。

- `accountId`: アカウント ID(オーナー / メンバー両方)
- `ownerAccountId`: 所属オーナー ID(オーナー本人は自身の id)
- `role`: 契約ロール(`accounts.role = admin / service_operator / end_user`)
- `isOwner`: オーナー判定(`accounts.is_owner=1`)
- `projectRoles`: `Map<projectId, 'admin' | 'member'>`。 `isOwner=true` のときは空 Map で構築され、認可ヘルパが bypass する

1.4.3 3.3 オーナー境界 (NFR-301)

すべての CRUD API は `owner_account_id` フィルタを必須 とする。`requireTenant / requireProject / requireInquiry` で境界チェックを実施し、他オーナーのデータには 404 を返す (漏洩防止のため 403 ではなく 404)。

1.4.4 3.4 不正アクセスの監査記録

境界違反検知時は `authz.owner_boundary_violation / authz.project_boundary_violation` を `retention_class=general` で記録する。詳細は [DD10_監査ログ書込・完全性検証.md](#) を参照。

1.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
権限設計	../02_基本設計/04_権限設計.md
認証・認可設計	../02_基本設計/08_認証・認可設計.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD01_アカウント・ユーザー管理.md / DD02_プロジェクト・FAQ 管理.md / DD10_監査ログ書込・完全性検証.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-017	iso-cross-owner-001	Isolation	workers/main-api/test/isolation/cross-owner.test.ts

1.6.1 5.1 オーナー境界によるデータ分離検証

```
// tests/isolation/cross-owner.test.ts
describe('クロス契約アクセス試行', () => {
  it('他契約の inquiry にアクセスできない', async () => {
    const ownerA = await createOwnerAccount();
    const ownerB = await createOwnerAccount();
    const inquiryB = await createInquiry(ownerB);
    const sessionA = await loginAs(ownerA.adminEmail);
    const res = await fetch(`/api/v1/inquiries/${inquiryB.id}`, { headers: { Cookie: sessionA } });
    expect(res.status).toBe(404); // 漏洩しないために 403 ではなく 404
  });
  // 50 ケース x 月次 + 5 ケース x 年次手動
});
```

1.6.2 5.2 その他観点

観点	内容
単体	requireTenant / requireProject / requireInquiry / requireRole の各分岐
結合	プロジェクト割当がない(account_project_grants 行なし)メンバーによる FAQ 編集試行で 404 偽装 (E-AUTHZ-PROJECT-DENIED)/ 該当 PJ の member+ なら 200
異常系	削除済オーナー所属メンバーのアクセス時 401 + セッション失効
境界値	v1.7 でアカウント削除はアカウントレベル物理削除に統一されたため、現存メンバーは常に 1 件以上の割当を保持。割当 0 件はテスト境界としては「データ不整合時のフェイルセーフ動作」 (/auth/login への強制ログアウト) を検証する
月次	クロス契約侵入試行 50 ケース月次 + 5 ケース年次手動

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済